Stochastic Anomaly Simulator

Project Documentation

Pole Anomaly Project

May 1, 2026

Contents

1 Introduction

This document outlines the architecture, setup, and execution modules of the Stochastic Anomaly Simulator pipeline. The project is designed to inject mathematically accurate, user-defined geometrically deformed anomalies (such as Squares, Exponential Curves, and Bell Curves) into synthetic radiation noise based on real-world sensor baselines.

2 Quick Setup Guide

This project isolates dependencies using a Python virtual environment to ensure absolute reproducibility.

2.1 Environment Setup

To initialize the project on a new machine, execute the following commands in the root of the project:

```
# 1. Create a Python Virtual Environment
python -m venv .venv

# 2. Activate the Environment (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 3. Install strictly required numerical libraries
pip install -r requirements.txt
```

2.2 Dependencies

The `requirements.txt` installs the following core standard math libraries:

- `numpy` & `pandas`: For fast mathematical array manipulations and template CSV handling.
- `plotly`: For high-fidelity, interactive visualizations saved in HTML logs.
- `scipy`, `statsmodels`, `tslearn`: Required specific dependencies for statistical interpolation and historical legacy dataset tests.

3 File Structure Review

The repository enforces a strict separation of concerns, divided into modular directories:

- `data/`: Located at the root, contains raw background sensor readings (e.g., `2015_months_DebitDos`) used as a baseline.
- `data-gen/`: The main pipeline directory containing the synthetic dataset generator and its components:
  - `data-gen/anomalies/`: Contains the normalized `.csv` blueprints (templates) for different anomaly shapes (e.g., `exp_square.csv`, `bell.csv`, `m_sq.csv`).

- `data-gen/data/`: The destination for the final generated output datasets (e.g., `synthetic_custom_dataset.csv`) utilized for neural network training.
- `data-gen/logs/`: The destination for Plotly's interactable HTML visualizations. Provides a pure geometric proof of concept without requiring external dashboard readers.
- `data-gen/src/`: Stores the pure mathematics and geometric generator libraries, decoupled from direct script logic.
- `data-gen/script/`: Stores execution pipelines and visualization launchers. Scripts utilize the math from `data-gen/src/` to generate datasets or debug plots.

4 Explanation of the Engine and Scripts

4.1 `data-gen/src/anomaly_generator.py`

This is the core physics computational engine. It provides the `generate_anomaly()` function which transforms a normalized CSV blueprint into a physical discrete array anomaly string. The engine mathematically scales the template temporally (period span) and vertically (amplitude intensity) while embedding Gaussian "deformation" limits declared in the CSV file logic.

4.2 `data-gen/script/generate_custom_dataset.py`

The final execution pipeline for training. It parses the real noise characteristics (Mean, Standard Deviation) from the original physical gamma sensor. Using a baseline simulation of 10,000 to 50,000 steps, it spontaneously samples anomaly templates from the library, calculates randomized injection amplitudes (e.g., 5x to 25x STD multipliers) and duration bounds, and seamlessly injects them into the dataset, tagging absolute ground-truth labels alongside cleanly tracking the pure anomaly wave.

4.3 `data-gen/script/test_generator_deformation.py`

A visual sandbox and boundary debugging tool. Iterating automatically over all discovered templates in the `data-gen/anomalies/` directory, it runs exactly 5 localized chaotic generation iterations of each template to visually verify and validate the mathematical variability limits enforced by the template's parameters. Exports directly to `anomaly_template_deformation_test.html`.

4.4 `data-gen/script/visualize_anomalies_templates.py`

This script simply visualizes the pure original mathematical blueprints. It renders the pure lines and graphs the interpolation properties exactly as described in the blueprint CSV to a localized HTML, without injecting randomization or temporal noise.

5 Anomalies Templates Creation

Instead of relying on rigid pre-recorded dataset distributions like legacy UCR databases, targeted geometric structures are freely drawn and mathematically described in the `data-gen/anomalies/` folder.

5.1 Configuration Blueprint (CSV Logic)

A standard anomaly template requires 7 strict column definitions mapping logic coordinates and behavioral constraints:
`time, value, t_minus, t_plus, v_minus, v_plus, interp`

- **time & value**: Standardized physical shape coordinates (normally scaling from 0.0 to 1.0). They map the major mathematical inflection milestones of the curve.
- **Deformation Multipliers (t_minus, t_plus, v_minus, v_plus)**: Defines spatial jitter limits per node. A value of `1.0` allows symmetric Gaussian distortion up to 100% of the global script variance parameter. A value of `0.0` creates an unyielding geometric wall perfectly locking the point in that specified space direction.
- **interp (Interpolation Mode)**: Defines the localized mathematical connection trajectory bridging the gap toward the targeted point.
  - `linear`: Connects sequential milestones with a simple Euclidean linear straight interpolation fallback.
  - `exp`: Instigates a smooth sluggish temporal acceleration morphing into a harsh geometric peak, useful for raw exponential growth burst characterizations.
  - `bell`: Executes a fractional phase-shift cosine envelope ($\cos(\pi x)$), generating an intensely organic smooth "S-curve". Ensures a perfect non-rigid mathematical slope shift suitable for defining Gaussian-like topographic bell shapes.